guard-track-branch-label

11 4 182

guard-track-branch-label.sh — § . . track+branch+alias label guard hook

Revision: 2 **Last modified:** 2026-07-26T12:18:28Z **Authority:** constitution submodule §11.4.182 (Track+branch work-stream identity label — extended 2026-07-26 with the `<model>` + `<effort>` fields) · §11.4.231 clause (F) (effort-tier) · §11.4.177 (inherited-by-reference) · §11.4.6 (no-guessing / honest boundary) **Classification:** universal (§11.4.17)

Overview

`scripts/hooks/guard-track-branch-label.sh` is a Claude Code **PreToolUse guard hook** that enforces §11.4.182: every agent-dispatching tool call (`Agent` / `Task` / `TaskCreate`) MUST carry a work-stream identity label at the START of its `description` (or, as a fallback, its `subagent` field), so parallel-track work (`/mnt/track1..4` , each on its own branch, each driven by a distinct claude-toolkit alias) is never ambiguous.

The label form was extended 2026-07-26 (§11.4.182 EXTENSION / §11.4.231 clause (F)) from the original 3-field `(T<N>/<branch> - <alias>)` to the 5-field `(T<N>/<branch> - <alias> - <model> - <effort>)` , where `<model>` records the model currently answering for that alias and `<effort>` the reasoning-effort setting in force (closed ladder `{low | medium | high | xhigh | max}`). The hook accepts **all three forms** — the 3-field legacy, the 4-field (`+ <model>`), and the 5-field (`+ <model> + <effort>`) — so a dispatcher still emitting a pre-model/pre-effort label is never blocked during migration.

The hook is inherited **by reference** (§11.4.177) — it lives in the constitution submodule and is wired into a consuming project's `.claude/settings.json` PreToolUse hooks. It is NEVER copied per project.

What it enforces

1. **FORMAT** — the label matches `^\(T([0-9]+|\?)/[^)]+ - [^)]+( - [^)]+){0,2}\) ` (a numeric-or-`?` track, a branch, `-` , an alias, then 0/1/2 optional `-` `<field>` segments for the `<model>` and `<effort>` fields, `)` , then a space). This admits the 3-, 4-, and 5-field forms; because each field atom is `[^)]+` (which itself can contain `-`), a label with even MORE trailing fields also matches — permissive by design, an extra field is never a block-cause. The atom is deliberately NOT tightened to a dash-excluding class: branch names AND aliases legitimately contain `-` (e.g. `feature/mistiq-vader` , `kimi-for-coding`), which such a class would wrongly REJECT (a §11.4.201(1) false-positive). A missing/malformed label on an agent dispatch is BLOCKED (exit 2).
2. **ALIAS-CORRECTNESS** (the §11.4.182 alias field) — a format-valid label is NOT sufficient. The label's `<alias>` (the field immediately after the track/branch segment, read positionally regardless of whether `<model>` / `<effort>` fields follow it) MUST match the **LIVE** alias derived from `CLAUDE_CONFIG_DIR` (basename `.claude-<alias>` → `<alias>`). A hand-typed / remembered / **stale** alias (e.g. `claude4` while the live session is `claude3`) is BLOCKED (exit 2) with the CORRECT label printed. The `<model>` and `<effort>` fields are NEVER cross-checked here (a model / effort can legitimately switch mid-turn, §11.4.6) — an honest `?` model or `?` effort, or an ABSENT model/effort field, is NEVER a block-cause.

The live alias is derived by **calling the reference labeler** `scripts/multitrack/track_branch_label.sh` — the single source of truth for the T/branch/alias derivation. The hook does not duplicate the derivation regex (DRY); the same labeler output drives both the alias check and the block message's "correct label" line.

Honest boundary (§ . .)

A `?` on **either** side is ACCEPTED, never fabricated into a verdict:

- **live alias unknown** (`CLAUDE_CONFIG_DIR` unset / non-`.claude-*`) → the hook cannot verify the alias, so any label is accepted on the alias axis;
- **caller honestly declares** `?` (`(T1/main - ?)`) → accepted even when the live alias is known.

The hook BLOCKS only a **concrete** alias that **provably disagrees** with a **known** live alias. `?` is the honest value, never a bluff.

Prerequisites

- `bash` (the hook and labeler are `#!/usr/bin/env bash`).
- `git` (optional — the labeler falls back to `?` for the branch if absent).
- The sibling labeler present at `scripts/multitrack/track_branch_label.sh` (a defensive inline fallback keeps the hook functional if it is unreachable).
- `CLAUDE_CONFIG_DIR` exported by the driving session (its basename encodes the alias). Absent ⇒ live alias `?` ⇒ alias axis not enforced (honest boundary).

Usage

Wired as a PreToolUse hook (matcher `Agent|Task|TaskCreate`) in `.claude/settings.json`:

```
{ "type": "command",
  "command": "bash \"$CLAUDE_PROJECT_DIR/constitution/scripts/hooks/guard-track-branch-label.sh\"" }
```

The hook receives the tool invocation as JSON on stdin and:

- **exit 0** → allow (Claude proceeds);
- **exit 2** → BLOCK; stderr is fed back to Claude as the refusal reason and includes the CORRECT label for this checkout+session so the caller can fix it;

- any other exit → non-blocking error (never used; the defensive fallback keeps the hook to 0/2).

Derive the exact label manually with:

```
bash constitution/scripts/multitrack/track_branch_label.sh   # e.g. (T1/main - claude3 - opus - high)
```

Edge cases

- **Non-agent tools** (`Bash` , `Read` , `Edit` , unknown tools) pass through untouched (exit 0) — the hook never breaks non-agent tools.
- **Empty / missing alias** (`(T1/main)` , `(T1/main - )`) → BLOCKED on FORMAT.
- **Extended forms** — the 4-field `(T1/main - claude3 - opus)` and 5-field `(T1/main - claude3 - opus - high)` labels are accepted; a `?` model / `?` effort, or an absent model/effort field, is never a block-cause.
- **Branch names with `-`** (e.g. `feature/mistiq-vader`) — a git ref cannot contain a space, so the `-` alias separator is unambiguous.
- **Labeler unreachable** — a defensive inline derivation mirrors the labeler so the hook still emits an actionable message and never exits non-0/2.

Internal behaviour

1. Read stdin JSON; extract `tool_name` (jq if present, else an awk fallback).
2. Non-agent tool ⇒ exit 0.
3. Extract the label from `description` (fallback `subagent`).
4. Call the labeler for the canonical `(T<N>/<branch> - <alias> - <model> - <effort>)` label; extract `_live_alias` from its alias field.
5. Extract `_dispatch_alias` from the caller's label.
6. Decide: FORMAT fails ⇒ BLOCK (format); FORMAT ok AND both aliases concrete AND they differ ⇒ BLOCK (alias); otherwise ⇒ exit 0.
7. On BLOCK, print the CORRECT label + how to fix; exit 2.

Tests / gates

- **Hermetic test:** `scripts/hooks/test_guard_track_branch_label.sh` (30 cases; controls `CLAUDE_CONFIG_DIR` per case for determinism — §11.4.50/ §11.4.98).
- **Pre-build gate:** `scripts/gates/cm_track_branch_label.sh` (`CM-TRACK-BRANCH-LABEL` — presence/exec/parseable + doc + behavioral alias-validation).
- **Paired §1.1 mutation:** `scripts/gates/cm_track_branch_label_mutation_test.sh` (a format-only hook that skips the alias check → the gate FAILs).

Related scripts

- `scripts/multitrack/track_branch_label.sh` — the reference labeler (single source of truth for the derivation).
- `scripts/hooks/guard-work-track-binding.sh` — the §11.4.191 work→track binding guard (sibling PreToolUse hook).
- `scripts/hooks/guard-forbidden-commands.sh` — the §11.4.109 anti-forgetting guard.

Last verified: 2026-07-26 (Rev 2 doc-sync for the §11.4.182 EXTENSION 5-field label; guard exit-code matrix re-verified on this checkout — 3/4/5/6-field format accept → 0, alias-mismatch → 2, no-label → 2, non-agent passthrough → 0).